

UF1844

INFORMÁTICA Y COMUNICACIONES



Certia
editorial

DESARROLLO DE APLICACIONES WEB EN EL ENTORNO SERVIDOR



Contenidos basados en los Certificados de profesionalidad

UF1844

DESARROLLO DE APLICACIONES WEB EN EL ENTORNO SERVIDOR



Certia
editorial

José Ramón Santos Dios es diplomado en Ciencias Empresariales por la Universidad de Vigo, graduado superior en TIC por la Universidad de Santiago de Compostela y máster en *Software Libre* por la UOC.

Desde el año 2003 se ha dedicado a la docencia, impartiendo acciones formativas relacionadas con las Tecnologías de la Información y las Comunicaciones en las principales empresas de formación de la comunidad autónoma de Galicia y como Agente TIC para la red CEMIT de la Xunta de Galicia.

En la actualidad, combina su profesión de docente con la elaboración de contenidos didácticos adaptados a diferentes niveles formativos y la elaboración de manuales relacionados con certificados de profesionalidad, también el desarrollo del diseño y programación web ocupan parte de su actividad profesional.

Para ver su currículum profesional completo puede acceder a el enlace de la red LinkedIn:

<https://www.linkedin.com/in/santosdios/>

Familia profesional: INFORMÁTICA Y COMUNICACIONES

Área profesional: Desarrollo

FICHA DE CERTIFICADO DE PROFESIONALIDAD (IFCD0210) DESARROLLO DE APLICACIONES CON TECNOLOGÍAS WEB (RD 1531/2011, de 31 de octubre modificado por el RD 628/2013, de 2 de agosto)

Correspondencia con el Catálogo Modular de Formación Profesional				
H. Q	Módulos certificado	H.CP	Unidades formativas	Horas
180	MF0491_3: Programación web en el entorno cliente	180	UF1841: Elaboración de documentos web mediante lenguajes de marca UF1842: Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guión UF1843: Aplicaciones técnicas de usabilidad y accesibilidad en el entorno cliente	60 90 30
240	MF0492_3: Programación web en el entorno servidor	240	UF1844: Desarrollo de aplicaciones web en el entorno servidor. UF1845: Acceso a datos en aplicaciones web del entorno servidor. UF1846: Desarrollo de aplicaciones web distribuidas.	90 90 60
90	MF0493_3: Implantación de aplicaciones web en entorno internet, intranet y extranet	90		90
	MP0391: Módulo de prácticas profesionales no laborales	80		
510	Duración horas totales certificado de profesionalidad	590	Duración horas módulos formativos	510

ÍNDICE

INTRODUCCIÓN.....	17
-------------------	----

UNIDAD DIDÁCTICA 1

1. El proceso de desarrollo de <i>software</i>	19
1.1. Modelos del ciclo de vida del <i>software</i>	21
1.1.1. En cascada (<i>waterfall</i>).....	21
1.1.2. Iterativo.....	22
1.1.3. Incremental.....	23
1.1.4. En V.....	24
1.1.5. Basado en componentes (CBSE).....	25
1.1.6. Desarrollo rápido (RAD).....	26
1.1.7. Ventajas e inconvenientes.	27
1.1.8. Pautas para la selección de la metodología más adecuada.....	27
1.2. Análisis y especificación de requisitos	29
1.2.1. Tipos de requisitos.....	30
1.2.2. Modelos para el análisis de requisitos	32
1.2.3. Documentación de requisitos	33
1.2.4. Validación de requisitos	35

1.2.5. Gestión de requisitos.....	35
1.3. Diseño.....	36
1.3.1. Modelos para el diseño de sistemas	37
1.3.2. Diagramas de diseño. El estándar UML.....	38
1.3.3. Documentación.....	39
1.4. Implementación. Conceptos generales de desarrollo de <i>software</i>	50
1.4.1. Principios básicos del desarrollo de <i>software</i>	50
1.4.2. Técnicas de desarrollo de <i>software</i>	52
1.5. Validación y verificación de sistemas	53
1.5.1. Planificación.....	54
1.5.2. Métodos formales de verificación	54
1.5.3. Métodos automatizados de análisis	56
1.6. Pruebas de <i>software</i>	57
1.6.1. Tipos	57
1.6.2. Pruebas funcionales (BBT).....	57
1.6.3. Pruebas estructurales (WBT)	59
1.6.4. Pautas de utilización	59
1.6.5. Diseño de pruebas	60
1.6.6. Ámbitos de aplicación.....	61
1.6.7. Pruebas de Sistemas.....	62
1.6.8. Pruebas de componentes.....	63
1.6.9. Automatización de pruebas. Herramientas.....	64

1.6.10. Estándares sobre pruebas de <i>software</i>	65
1.7. Calidad del <i>software</i>	66
1.7.1. Principios de calidad del <i>software</i>	67
1.7.2. Métricas y calidad del <i>software</i>	68
1.7.3. Concepto de métrica y su importancia en la medición de la calidad	68
1.7.4. Principales métricas en las fases del ciclo de vida <i>software</i>	68
1.7.5. Estándares para la descripción de los factores de Calidad	71
1.7.6. ISO-9126	72
1.8. Herramientas de uso común para el desarrollo de <i>software</i>	76
1.8.1. Editores orientados a lenguajes de programación	77
1.8.2. Compiladores y enlazadores	78
1.8.3. Generadores de programas	79
1.8.4. Depuradores. De prueba y validación de software. Optimizadores de código. Empaquetadores.....	80
1.8.5. Generadores de documentación de <i>software</i>	82
1.8.6. Gestores y repositorios de paquetes. Versionado y control de dependencias.....	82
1.8.7. De distribución de <i>software</i>	83
1.8.8. Gestores de actualización de <i>software</i>	83
1.8.9. De control de versiones.	83
1.8.10. Entornos integrados de desarrollo (IDE) de uso común.....	85
1.9. Gestión de proyectos de desarrollo de <i>software</i>	86

1.9.1. Planificación de proyectos	87
1.9.2. Control de proyectos	88
1.9.3. Ejecución de proyectos	89
1.9.4. Herramientas de uso común para la gestión de proyectos	91

UNIDAD DIDÁCTICA 2

2. La orientación a objetos.....	93
2.1. Principios de la orientación a objetos	98
2.1.1. Comparación con la programación estructurada.....	99
2.1.2. Ocultación de información (<i>informationhiding</i>)	100
2.1.3. El tipo abstracto de datos (ADT).....	100
2.1.4. Encapsulado de datos.....	101
2.1.5. Paso de mensajes.....	102
2.2. Clases de objetos	102
2.2.1. Atributos, variables de estado y variables de clase	103
2.2.2. Métodos y requisitos e invariantes.....	106
2.2.3. Gestión de excepciones.....	108
2.2.4. Agregación de clases.....	109
2.3. Objetos	110
2.3.1. Creación y destrucción de objetos	110

2.3.2.	Llamada a métodos de un objeto	111
2.3.3.	Visibilidad y uso de las variables de estado	112
2.3.4.	Referencias a objetos	115
2.3.5.	Persistencia de objetos	115
2.3.6.	Optimización de memoria y recolección de basura (<i>garbagecollection</i>)	116
2.4.	Herencia.....	117
2.4.1.	Concepto de herencia. Superclases y subclases.....	118
2.4.2.	Herencia múltiple.....	119
2.4.3.	Clases abstractas.....	120
2.4.4.	Tipos de herencia.....	121
2.4.5.	Polimorfismo y enlace dinámico (<i>dynamicbinding</i>)	125
2.4.6.	Directrices para el uso correcto de la herencia.....	126
2.5.	Modularidad.....	127
2.5.1.	Librerías de clases. Ámbito de utilización de nombres	127
2.5.2.	Ventajas de la utilización de módulos o paquetes	129
2.6.	Genericidad y sobrecarga.....	130
2.6.1.	Concepto de genericidad	130
2.6.2.	Concepto de Sobrecarga. Tipos de sobrecarga	131
2.6.3.	Comparación entre genericidad y sobrecarga.....	131
2.7.	Desarrollo orientado a objetos	132
2.7.1.	Lenguajes de desarrollo orientado a objetos de uso común .	132

2.7.2. Herramientas de desarrollo	134
2.8. Lenguajes de modelización en el desarrollo orientado a objetos	134
2.8.1. Uso del lenguaje unificado de modelado (UML) en el desarrollo orientado a objetos.....	135
2.8.2. Diagramas para la modelización de sistemas orientados a objetos.....	137

UNIDAD DIDÁCTICA 3

3. Arquitecturas web	141
3.1. Concepto de arquitectura web	143
3.2. El modelo de capas.....	144
3.3. Plataformas para el desarrollo en las capas servidor	145
3.4. Herramientas de desarrollo orientadas a servidor de aplicaciones web	147
3.4.1. Tipos de herramientas.....	147
3.4.2. Extensibilidad. Instalación de módulos.....	149
3.4.3. Técnicas de configuración de los entornos de desarrollo, preproducción y producción	150
3.4.4. Funcionalidades de depuración.....	151

UNIDAD DIDÁCTICA 4

4. Lenguajes de programación de aplicaciones web en el lado servidor.....	153
4.1. Características de los lenguajes de programación web en servidor...	155
4.2. Tipos y características de los lenguajes de uso común.....	158
4.2.1. Interpretados orientados a servidor.....	159
4.2.2. Lenguajes de cliente interpretados en servidor	159
4.2.3. Lenguajes compilados	160
4.3. Criterios en la elección de un lenguaje de programación web en servidor. Ventajas e inconvenientes.....	161
4.4. Características generales.....	162
4.4.1. Tipos de datos	163
4.4.2. Operadores básicos. Manipulación de cadenas de caracteres	166
4.4.3. Estructuras de control. Bucles y condicionales.....	170
4.4.4. Módulos o paquetes.....	172
4.4.5. Gestión de bibliotecas (<i>libraries</i>).....	173
4.5. Gestión de la configuración	173
4.5.1. Configuración de descriptores	174
4.5.2. Configuración de ficheros	175
4.6. Gestión de la seguridad.....	175

4.6.1.	Conceptos de identificación, autenticación y autorización...	177
4.6.2.	Técnicas para la gestión de sesiones	178
4.7.	Gestión de errores	180
4.7.1.	Técnicas de recuperación de errores.....	181
4.7.2.	Programación de excepciones.....	182
4.8.	Transacciones y persistencia.....	183
4.8.1.	Acceso a bases de datos. Conectores	184
4.8.2.	Estándares para el acceso a bases de datos	185
4.8.3.	Gestión de la configuración de acceso a bases de datos.....	187
4.8.4.	Acceso a directorios y otras fuentes de datos.....	188
4.9.	Componentes en servidor.....	189
4.9.1.	Ventajas e inconvenientes en el uso de contenedores de componentes.....	190
4.10.	Modelos de desarrollo	191
4. 10.1.	El modelo vista controlador.....	192
4. 10.2.	Modelo: programación de acceso a datos	193
4. 10.3.	Vista: desarrollo de aplicaciones en cliente	194
4. 10.4.	Eventos e interfaz de usuario	195
4. 10.5.	Programación del controlador	196
RESUMEN FINAL.....		197
ACTIVIDADES		199

AUTOEVALUACIÓN FINAL.....	219
SOLUCIONES	233
BIBLIOGRAFÍA	235

INTRODUCCIÓN

El manual «UF1844. Desarrollo de aplicaciones web en el entorno servidor» forma parte del módulo formativo «MF0492_3: Programación web en el entorno servidor», del Catálogo Nacional de Cualificaciones Profesionales, y pertenece al certificado de profesionalidad «IFCD0210: Desarrollo de aplicaciones con tecnologías web».

El manual tiene por objetivo aprender los fundamentos programación web y desarrollo de software, conocer las arquitecturas y lenguajes en la construcción de aplicaciones web, las herramientas de las que disponemos para desarrollar el código de programación, las librerías de código, los paquetes y módulos de las aplicaciones, y conocer la aplicación de estos principios por medio de las diferentes metodologías que disponemos a la hora de realizar el desarrollo de aplicaciones web.

Finalmente, encontraremos diferentes actividades y pruebas de evaluación para poner en práctica los contenidos aprendidos.

Este manual te permitirá adquirir las siguientes capacidades y criterios de evaluación:

- C1: Crear componentes software con tecnologías de desarrollo orientadas a objetos.
 - o CE1.1 En un supuesto práctico en el que se pide gestionar componentes software en el entorno del servidor mediante herramientas de desarrollo y lenguajes de programación contando con documentación de diseño detallado:
 - Crear y archivar componentes software.
 - Modificar y eliminar componentes software.
 - Depurar y verificar los componentes software elaborados.
 - o CE1.2 Crear objetos, clases y métodos adecuados a la funcionalidad del componente software a desarrollar utilizando lenguajes de programación orientados a objetos.

- o CE1.3 Formular estructuras de datos y flujo de control mediante lenguajes de programación orientados a objetos según la funcionalidad del componente software a desarrollar.
- o CE1.4 Documentar el componente software desarrollado.
- o CE1.5 En un supuesto práctico en el que se pide construir componentes de software mediante herramientas de desarrollo y lenguajes de programación orientados a objetos a partir de documentación de diseño detallado:
 - Integrar componentes software de control del contenido de los documentos ubicados en el servidor para ser utilizados en el entorno del cliente tipo servlet.
 - Integrar gestión de ficheros en el componente software a desarrollar.
 - Integrar gestión de errores en el componente software a desarrollar.
 - Utilizar variables de servidor en el componente software a desarrollar para proporcionar acceso a las propiedades del servidor.
 - Integrar seguimiento de sesiones de usuario y propiedades de la aplicación web a desarrollar en el componente software a construir.
 - Crear componentes software con la funcionalidad de aplicación de cliente para ser utilizado en el entorno cliente tipo applet.
 - Crear componentes software que puedan ofrecer su funcionalidad a otros componentes software del mismo servidor u otros servidores de la red.

CONTENIDO

- 1.1. Modelos del ciclo de vida del *software*
- 1.2. Análisis y especificación de requisitos
- 1.3. Diseño
- 1.4. Implementación. Conceptos generales de desarrollo de *software*
- 1.5. Validación y verificación de sistemas
- 1.6. Pruebas de *software*
- 1.7. Calidad del *software*
- 1.8. Herramientas de uso común para el desarrollo de *software*
- 1.9. Gestión de proyectos de desarrollo de *software*

El proceso del desarrollo de *software* es una disciplina que establece un marco de referencia para generar los procesos, tareas y actividades involucradas en todo el desarrollo, explotación y mantenimiento de un producto o aplicación de *software*, desde su planificación inicial y definición de requisitos, pasando por el desarrollo y codificación del producto, hasta la finalización del mismo. La idea fundamental de esta disciplina es definir un modelo de ciclo de *software* y aplicarlo a determinados proyectos, mediante este ciclo de vida se especifica un enfoque general que planifica y desarrolla una serie de etapas por la que pasa el producto y que lleva asociada una serie de tareas y actividades que se deben desarrollar en un orden específico.

1.1. Modelos del ciclo de vida del *software*

A lo largo de las últimas décadas se han desarrollado diferentes modelos de desarrollo de *software*. Este tipo de modelos han evolucionado con los nuevos desarrollos de *software* que han aparecido durante todo este tiempo, entre los diferentes modelos de desarrollo podemos citar los que se presentan a continuación.

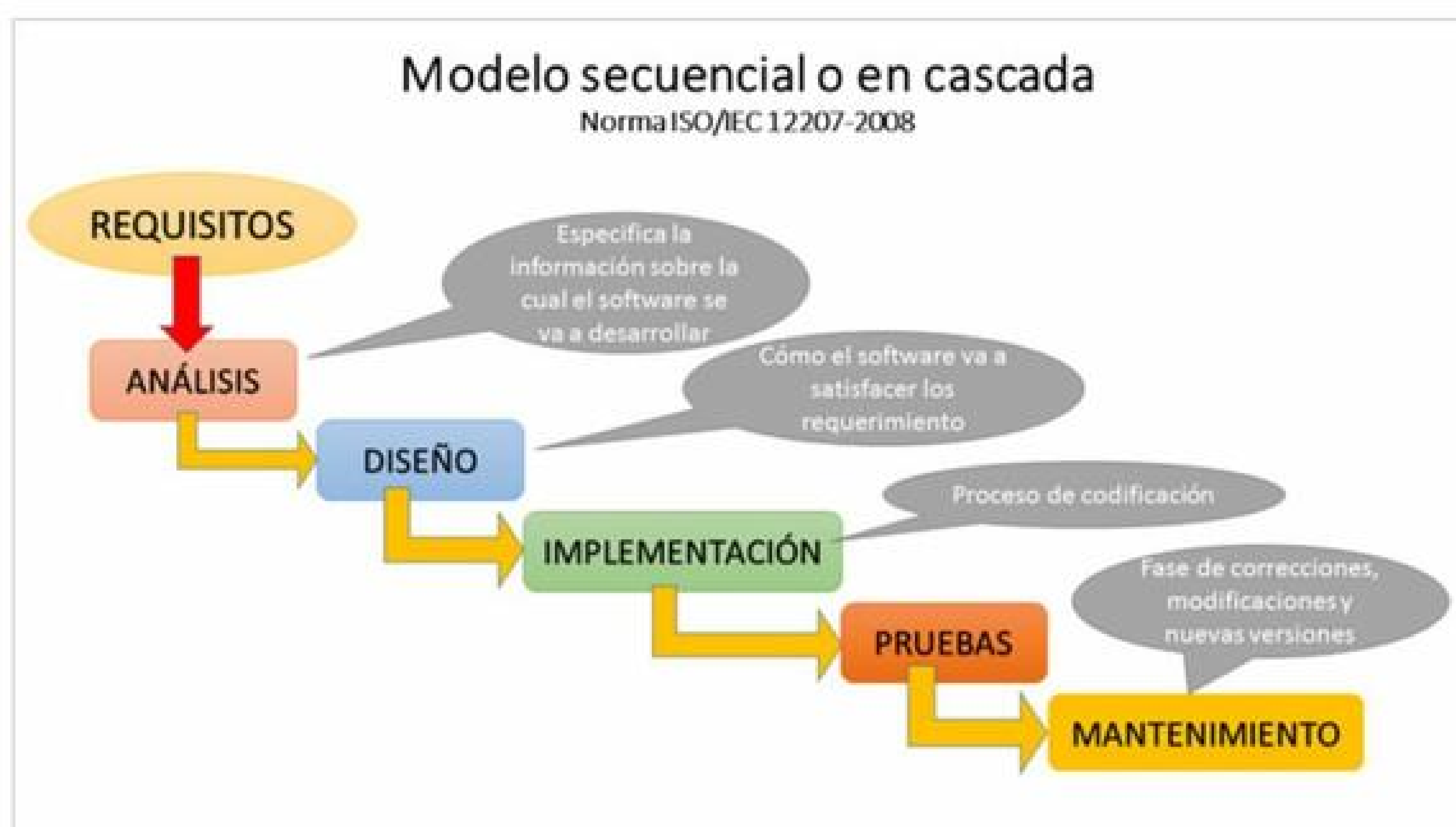
1.1.1. En cascada (*waterfall*)

Es también conocido como el modelo lineal-secuencial o ciclo de vida clásico, ya que fue el primero en aparecer y utiliza la metodología más clásica en el desarrollo de *software*. Consiste en un conjunto de etapas secuenciales donde cada una de ellas no puede comenzar hasta que termine la anterior y se revise mediante parte de los miembros del equipo y se genere la documentación pertinente. Estos documentos o informes se utilizarán como base para las siguientes etapas de desarrollo.

Para desarrollar cada una de estas etapas, se definirán previamente un conjunto de requisitos mediante una planificación por parte del equipo de desarrollo.

Desventajas del modelo en cascada:

- Es un modelo poco flexible, ya que los requisitos se realizan al inicio de la planificación del proceso, y estos requisitos pueden ir cambiando por parte del cliente o el entorno tecnológico.
- Una vez que se ha completado toda una etapa resulta difícil volver atrás en el proyecto, ya que el resto de los procesos dependen de esta.
- En muchos proyectos es muy difícil obtener todos los requisitos al comienzo del mismo.



1.1.2. Iterativo

Las limitaciones del modelo en cascada han llevado un nuevo modelo iterativo para mejorar los problemas del establecimiento inicial de los requisitos del proyecto. Este modelo consiste en la repetición del ciclo en cascada de forma iterativa, revisando los requisitos durante todo el proceso mediante una sucesión de versiones mejoradas. Cada iteración será revisada por los usuarios o clientes del proyecto para realizar propuestas de mejoras.

Las ventajas de optar por este modelo frente al modelo clásico en cascada están en la posibilidad de visualizar los resultados a más corto plazo y mejorar el *feedback* con el cliente, por lo que aumenta las posibilidades de satisfacción del producto por parte de los usuarios y, en consecuencia, es más adecuado para los proyectos centrados en el usuario que el anterior modelo.

A las fases del anterior modelo debemos añadir las siguientes medidas para implementar el modelo iterativo:

- **Planificación:** establecer las alternativas de desarrollo analizando las restricciones de cada una de las alternativas.
- **Análisis de riesgos:** establecer los riesgos de cada una de las alternativas y las formas de resolverlos.
- **Desarrollo del producto:** mediante cada iteración se establecerá la evaluación por parte del cliente y la revisión del modelo y la previsión de los recursos necesarios para la siguiente fase, para ver si se cumplen los objetivos especificados.

Entre las desventajas que nos podemos encontrar en este modelo está el establecimiento de los requisitos para pasar de fase, la dificultad de la evaluación de riesgos, el riesgo de la duración demasiado prolongada del proyecto y el cálculo del presupuesto del mismo.

1.1.3. Incremental

El siguiente modelo combina los elementos del modelo iterativo con el de cascada, mediante la sucesión iterativa de secuencias que se definen al principio del proyecto. Al final de cada ciclo se produce una entrega de la versión parcial del producto que se irá incrementando con una determinada frecuencia respecto a las anteriores hasta llegar al producto final.

La ventaja respecto al anterior modelo es la disponibilidad por parte del cliente de disponer de un producto de *software* incompleto que pueden ir testeando y añadiendo mejoras en los requisitos que se implementarán en el siguiente ciclo.

En este tipo de modelos cobra especial importancia el desarrollo de prototipos, aunque no es exclusivo de este modelo.

El prototipo es una versión limitada del producto a desarrollar que permite a usuarios y desarrolladores realizar pruebas y testeos en situaciones reales, explorando su uso para hacerse una idea del funcionamiento del mismo. Mediante este sistema posibilitamos la validación y el refinamiento de los requisitos del proyecto. El prototipo supone un método claro para definir el resultado final de manera concreta. El tipo de prototipo más habitual es el de interfaz de

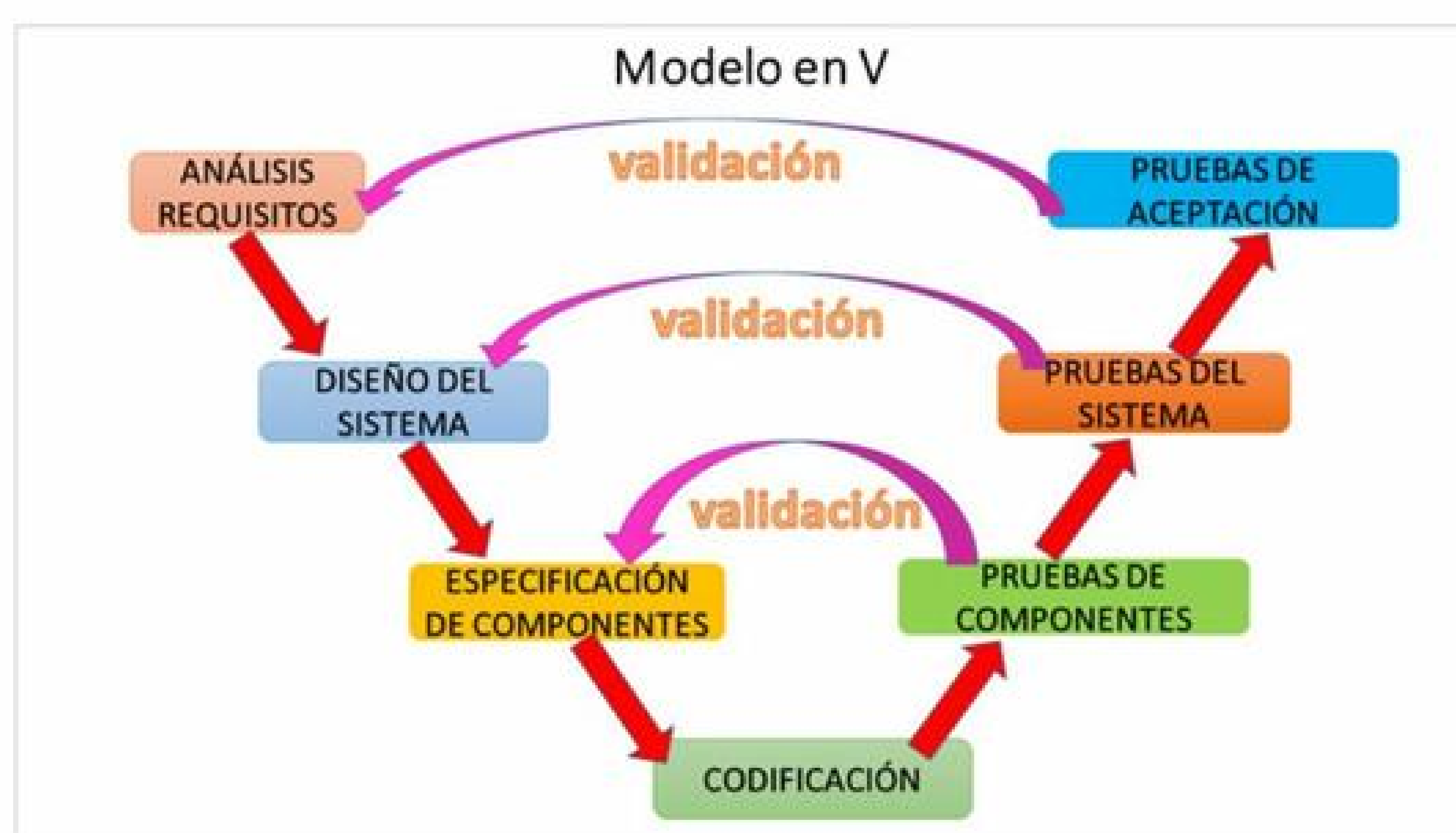
usuario, otro tipo de prototipos utilizados es el funcional, que sirve para evaluar determinados procesos antes de tomar decisiones sobre su diseño.

La ventaja de utilizar los prototipos es la rápida visualización de los resultados del producto final, el análisis de la interfaz de usuario y sus funcionalidades, reduciendo los riesgos de construir productos que no satisfagan las necesidades de los usuarios.

1.1.4. En V

El modelo en V es una variación del modelo clásico en cascada que describe las fases del proceso y los resultados que deben ser producidos, de manera que en la parte izquierda se representan los requisitos del proyecto y las especificaciones del mismo, mientras que en el derecho se representa la verificación e integración de dichos requisitos. La V hace referencia a los conceptos de validación y verificación que se implementarán en cada una de las fases del proyecto.

De esta manera, aunque las etapas son similares a las del modelo en cascada, cada una de dichas fases tendrá un proceso de validación, verificación y pruebas.



Como podemos ver en la ilustración, el proceso de desarrollo en V define las pruebas que se realizarán en cada una de las fases, además de indicar a qué fase se debería regresar en el caso de que se encontrarán fallos en los testeos correspondientes.

1.1.5. Basado en componentes (CBSE)

El modelo basado en componentes trata de abordar el problema de la creciente complejidad en los proyectos de *software* y el tiempo de adaptación a los cambios, mediante el desarrollo de aplicaciones basadas en el ensamblado de módulos de *software* reutilizables, diseñados previamente independientemente de las aplicaciones en los que serán utilizados.

Por lo tanto, el elemento fundamental en este tipo de desarrollo es el componente de *software*, que consiste en un módulo de programación independiente que puede ser incorporado a diferentes tipos de proyectos. Los componentes se distinguen por una serie de características:

- **Aislados:** se pueden reutilizar de manera independiente en una plataforma.
- **Compuestos:** puede a su vez ser compuesto por otros componentes más pequeños para formar otros componentes o aplicaciones.
- **Opacos:** no es necesario acceder a sus detalles internos para utilizarlos.



Un conjunto de componentes trabajando juntos se distribuyen como paquetes de *software*, suelen contener metainformación sobre cómo utilizar el paquete por parte de los desarrolladores.

Esta metainformación contiene la siguiente información:

- Define de manera exacta como se utiliza el componente y sus interfaces.

- Define el modo de interacción entre componentes y su ensamblado.
- Define el servicio de soporte, gestión de su ciclo de vida y versiones.

Entre las ventajas de utilizar este modelo de desarrollo están la mejora de la productividad mediante la reutilización de *software*, disminución de la complejidad en el desarrollo del *software*, mejora de los tiempos de desarrollo, incremento de la calidad del *software*, mejora del mantenimiento, etc.

1.1.6. Desarrollo rápido (RAD)

El desarrollo rápido de aplicaciones (RAD, *Rapid Application Development*) pertenece a un conjunto de métodos de desarrollo denominado metodologías ágiles, y establece un método de desarrollo iterativo que hace hincapié en el desarrollo rápido de prototipos y la entrega para el testeo y retroalimentación de los requisitos del proyecto. Para ello, se suele servir de herramientas de desarrollo rápido de prototipos por medio de plataformas de programación visuales, como Visual Studio, Lazarus, Gambas, Delphi, Foxpro, etc.

Las etapas en este tipo de desarrollo son las siguientes:

1. **Modelo de gestión:** en primer lugar, diseñamos el flujo de la información entre los diferentes módulos y el papel de cada uno de ellos respecto a esa información.
2. **Modelo de datos:** se definen las propiedades y atributos de los datos que manejará este flujo de información entre componentes.
3. **Modelo de proceso:** describe los procesos que se crearán para tratar los datos definidos en la anterior fase y el flujo de estos con los módulos, se crean descripciones de como añadir, suprimir, modificar, los datos.
4. **Construcción del prototipo:** se ensamblarán los componentes por medio de herramientas automáticas de generación de aplicaciones.
5. **Fase de pruebas:** aunque los componentes ensamblados ya han sido comprobados previamente se realizará una comprobación general de todo del proyecto.

Uno de los desarrollos rápidos más utilizados es la metodología Scrum, que

se basa en iteraciones de 30 días para producir código revisable por medio de entrevistas y reuniones cortas y frecuentes donde se acuerda una lista priorizada de tareas.

Entre las ventajas de utilizar este tipo de desarrollo podemos citar la capacidad para medir y evaluar de manera eficaz el progreso del desarrollo, la generación rápida de código y prototipos, el uso de componentes reutilizables, lo que permite la iteración y retroalimentación constante con los usuarios, y una evaluación del desarrollo en todo momento.

1.1.7. Ventajas e inconvenientes

Cada uno de los modelos de desarrollo anteriormente citados posee un conjunto de ventajas e inconvenientes dependiendo del tipo de proyecto a desarrollar por lo tanto la elección de uno u otro dependerá de las características y particularidades del proyecto.

Los métodos de desarrollos ágiles o iterativos poseen ciclos de vida de corta duración que favorecen la comunicación entre clientes y usuarios con los desarrolladores, pero también dependen de la habilidad por parte del equipo de responder a los cambios que se produzcan en el proceso y de disponer de los recursos necesarios para tales cambios.

El tipo de presupuesto o contrato que se realice para el proyecto también es determinante en la elección de un modelo u otro, los contratos poco flexibles o prefijados son más adecuados para el modelo tradicional, también el grado de interacción del cliente, con usuarios o clientes poco dispuestos a revisar continuamente los prototipos y versiones se hace necesario utilizar el modelo en cascada.

1.1.8. Pautas para la selección de la metodología más adecuada

Existen diferentes métodos para elegir una u otra metodología en cada caso, dependiendo del tamaño y complejidad del proyecto, el equipo de trabajo, el tipo de cliente y usuario, se dará unas u otras condiciones para elegir la metodología adecuada. La principal distinción que podemos hacer a la hora de elegir una metodología será entre las tradicionales y las metodologías ágiles, pues estos dos grupos están pensados para diferentes tipos de proyectos y circunstancias. En la siguiente tabla podemos ver las pautas que debemos considerar para cada una

de ellas:

Metodologías tradicionales	Metodologías ágiles
Se basan en estándares y normas elaboradas por estos	Se basan en heurísticas originadas en prácticas de producción
Resistencia a los cambios durante el proceso	Adaptables a los cambios durante el proceso
Proceso muy controlado con numerosas normas	Proceso poco controlado con pocas normas
Contrato rígido y prefijado. Funciona mejor en entornos de precio cerrado.	Contrato flexible y adaptable. En entornos de precio cerrado se corre el riesgo de incurrir en pérdidas.
Menos interactividad con el cliente	El cliente es parte del desarrollo e interactúa frecuentemente con reuniones
Ejecuta las etapas una sola vez y estas son inamovibles, hasta que no se finaliza una etapa con éxito no se pasa a la siguiente	Las etapas se ejecutan de manera cíclica, revisión constante de cada etapa
Altos costes al implementar algún cambio	Costes más elevados al comienzo del proyecto
El cliente no puede acceder al producto hasta el final del proceso, no existe tanta dependencia de la disponibilidad del cliente para el desarrollo	El cliente revisa el producto en cada fase del proceso, requiere mayor disponibilidad del cliente
Abundante documentación del proyecto	Documentación escasa
Planificación y un diseño más sencillos y directos	Planificación más compleja, diseños cambiantes
Proceso más fácil de medir	Proceso más complicado de medir
El cliente solamente puede revisar el resultado al final del proceso	Mejores posibilidades de anticipación a la corrección de fallos o requisitos del cliente
Mayor certidumbre en el cumplimiento de los plazos	Riesgo de no cumplir los plazos establecidos, debido a los continuos cambios en el proyecto

1.2. Análisis y especificación de requisitos

La definición correcta de requisitos del proyecto de *software* es esencial para un desarrollo exitoso del producto. De lo contrario, se generarán una gran cantidad de problemas como el coste de rectificación, solventar problemas en las etapas más avanzadas o complejas, problemas derivados del cumplimiento de plazos, etc. Por tanto, debemos analizar a fondo cada uno de estos requisitos, documentándolos adecuadamente y comprobando su cumplimiento en cada una de las etapas del proceso.

Los procesos de desarrollo del *software* están sujetos a diferentes influencias como los cambios en los requisitos, los costes o los plazos estimados, dificultades técnicas de producción, estos procesos tendrán una serie de objetivos medibles como la entrega en plazo, los costes estimados, los niveles de calidad, y finalmente los riesgos asociados.

El análisis de requisitos es un proceso formado por un conjunto de actividades cuyo principal objetivo es generar un documento, denominado «Documento de especificación de requisitos de *software*», que nos permita validar las acciones que se tendrán que llevar a cabo para realizar el proyecto, programar estas actividades y conocer las herramientas y equipo de trabajo que se utilizarán para llevarlas a cabo.

Dicho análisis de requisitos se dividirá en diferentes fases:

1. **Obtención de requisitos:** mediante diferentes instrumentos como entrevistas, observación sobre el funcionamiento del entorno del cliente y su actividad, análisis de documentación en proyectos similares, realización de cuestionarios al personal implicado en el proyecto, etc., se realizará una recopilación.
2. **Clasificación y organización de requisitos:** en esta fase se recopilan todos los requisitos que se han recogido en la fase anterior y se clasifican según criterios determinados.
3. **Ordenación de requisitos:** se realizará un filtrado y agrupación de requisitos por prioridad.
4. **Documentación de requisitos:** se documentarán detalladamente todos

los requisitos y se elaborará un informe formal que servirá como guía para el cumplimiento de dichos requisitos.

1.2.1. Tipos de requisitos

Cuando describimos un requisito, debemos tener en cuenta una serie de especificaciones que concreten detalladamente su funcionalidad. Para ello, debe ser identificado, clasificado y organizado mediante referencias cruzadas a otros documentos que lo relacionan. Podemos clasificar los tipos de requisitos del siguiente modo:

- **Requisitos funcionales:** tratan sobre los servicios que el producto debe realizar, por lo que especificarán entradas y salidas de datos. Estos requisitos deberán relatar todos los servicios demandados por el usuario de manera coherente.
- **Requisitos no funcionales:** recogen las limitaciones o tareas que el producto no puede realizar, este tipo de requisitos describen la disponibilidad del rendimiento del producto, capacidad de almacenamiento o memoria, incompatibilidades, etc.
- **Requisitos del dominio:** indican propiedades y limitaciones del dominio del producto.
- **Requisitos del producto:** este conjunto de requisitos detalla características propias del producto, como su eficiencia, usabilidad, tiempos de respuesta, etc.
- **Requisitos de organización:** describen procedimientos y políticas de la organización del cliente, como los detalles de la entrega, el cumplimiento de algún estándar de calidad o la implementación del producto en la organización.
- **Requisitos externos:** engloban los requisitos que pertenecen a factores externos al producto, como las normas legales, o normas de la organización del cliente.
- **Requisitos del usuario:** engloban los requisitos que necesita cumplir el producto para que el usuario pueda interactuar con el producto sin poseer conocimientos técnicos sobre dicho producto.

- **Requisitos del sistema:** otra forma de analizar los requisitos del usuario es englobarlos dentro de los requisitos del sistema, que describirá los requisitos en cuanto al comportamiento y restricciones de este con el usuario
- **Requisitos específicos:** este tipo de requisitos tienen enorme importancia, ya que describen detalles concretos de cumplimiento del *software*. Por lo tanto, deben estar especificadas con descripción muy detallada, describiendo las entradas, salidas y funciones de cada procedimiento.
- **Requisitos de interfaz externa:** este apartado describirá de manera minuciosa cada una de las entradas y salidas del *software*, detallando cada uno de los elementos de la interface, su propósito, la naturaleza de los datos de entrada y salida, tipos de unidades de medida utilizadas y su relación con otras entradas y salidas del sistema, así como los mensajes de la interfaz en la interacción con el usuario.
- **Requisitos lógicos de la base de datos:** este conjunto de requisitos especifica toda la información guardada en una base de datos, como el tipo de información procesada, las entidades de datos y sus relaciones, etc.
- **Restricciones de diseño:** en este grupo de requisitos englobamos los límites para realizar el diseño del producto, relacionados con la normativa o estándares existentes, las condiciones del cliente su entorno o cualquier otra circunstancia que los limite.

Otro tipo de requisitos vienen dados por atributos inherentes al desarrollo del *software* y que están relacionados con la calidad del producto. Este tipo de propiedades son denominadas atributos del *software* y entre ellos podemos citar los siguientes:

- **Fiabilidad:** describen la capacidad del producto para desempeñar sus funciones durante un período de tiempo.
- **Seguridad:** describe los factores que influyen en la protección de los datos del *software* en su manejo por parte del usuario, utilizando procedimientos de seguridad como el cifrado de datos, protección de las comunicaciones, comprobaciones de la integridad de los datos, copias de

seguridad, validación de usuario mediante *login*, etc.

- **Usabilidad:** describe cuestiones relacionadas con la facilidad de uso del *software* por parte del usuario.
- **Mantenimiento:** describe los aspectos relacionados con la conservación del *software* a lo largo del tiempo.

1.2.2. Modelos para el análisis de requisitos

Los modelos de análisis de requisitos nos proporcionan una serie de pautas para elaborar los documentos estos modelos suelen incluir una serie de instrumentos, recomendaciones o herramientas para analizar y supervisar adecuadamente los requisitos del proyecto.

Algunos de estos instrumentos son los siguientes:

- **Descripción detallada del producto a realizar:** ayudará a mejorar la información y documentación de requisitos a realizar para su realización.
- **Elaboración de métricas:** mejoran el análisis de requisitos específicos, proporciona pautas para el cumplimiento de dichos requisitos y procesos para llegar a cumplir esos requisitos.
- **Definición del alcance de requisitos:** ayudará a definir la importancia del requisito, los recursos necesarios para cumplirlo y la relación del requisito con los otros.
- **Definición de tareas asociadas a cada requisito:** indicando una detallada descripción de los procesos necesarios para realizarlo, las limitaciones y restricciones y la manera de testarlos.
- **Calendario de ejecución de las tareas a realizar para cumplir los requisitos:** será esencial para cumplir los objetivos del proyecto y la entrega a tiempo al cliente.
- **Documentación y referencias del proyecto:** se deben detallar las fuentes, autor, fechas de elaboración, mediante un apéndice u otros documentos.

El modelo finalmente elegido deberá apoyarse en una serie de documentos

y estándares que garanticen su cumplimiento y unos estándares mínimos de calidad del producto. Para ello, podemos utilizar los siguientes recursos:

- Documentación asociada a las pruebas de *software*.
- Descripción del diseño del *software*.
- Plan de aseguramiento de la calidad del *software*.
- Estándar de guía para el desarrollo de especificación de requisitos del sistema.

1.2.3. Documentación de requisitos

La documentación de requisitos es esencial para notificar a cada una de las partes participantes en el proyecto del nivel de cumplimiento de dichos requisitos. En esta documentación se detallarán todas las propiedades que el producto debe poseer y el conjunto de restricciones que este debe cumplir. Toda esta información está dirigida a los usuarios finales, los desarrolladores y cualquier otro agente vinculado en el proyecto.

El documento final suele ser denominado «Documento de especificación de requisitos (ERS)» y debe tener un formato estándar siguiendo la línea de productos similares elaborados por la organización para el cliente. De este modo, será más fácil su seguimiento, comprensión y verificación por parte de los desarrolladores y clientes del proyecto.

Existe un estándar comúnmente utilizado por las grandes empresas, la norma ANSI/IEEE 830-1993 la cual propone una estructura para el documento como la siguiente:

1. Introducción: Esta sección relatará una descripción global del contenido del documento, describiendo su propósito, ámbito, definiciones, referencias y cualquier otra información general sobre el documento, como la descripción relativa al comportamiento del *software*, el problema que resuelve dicho producto, las pautas de validaciones donde se incluyen diversas pruebas a realizar al producto sobre su rendimiento, etc.

1.1. Propósito del documento de requisitos: se definirá la finalidad del documento y a quien va dirigido, documentando las interfaces,

funcionalidad, requisitos de rendimiento, bases de datos, restricciones de diseño, etc.

1.2. **Ámbito de aplicación del producto:** debemos especificar toda la información relativa al ámbito de implementación del producto y la integración o no en otros sistemas de la organización del cliente.

1.3. **Definición, acrónimos y abreviaturas:** debemos especificar un glosario de términos explicando cada uno de los términos confusos o tecnicismos utilizados.

1.4. **Referencias:** se describirá toda la documentación de apoyo, normas de uso, estándares y otras normas utilizadas para la elaboración del documento.

1.5. **Descripción general del resto del documento.**

2. Descripción general:

2.1. **Perspectiva del producto:** idea general del producto y sus funcionalidades.

2.2. **Funciones del producto:** concepto y definición de los requisitos funcionales y no funcionales del sistema, restricciones y desarrollo del funcionamiento del producto.

2.3. **Características de los usuarios:** descripción del perfil del usuario final del producto, sus conocimientos acerca de la tecnología que engloba el producto.

2.4. **Limitaciones generales:** costes y problemas asociados al desarrollo del proyecto.

2.5. **Suposiciones y dependencias:** indicaremos las diferentes dependencias entre las secciones del documento, el orden de lectura

3. Los requisitos específicos: que cubren los requisitos funcionales, no funcionales y de interfaz y que veremos más adelante:

3.1. **Requisitos funcionales.**

3.2. **Requisitos de interfaz externa.**

3.3. Requisitos de ejecución.

3.4. Requisitos de diseño.

3.5. Atributos de calidad.

1.2.4. Validación de requisitos

Esta fase comprueba la calidad del producto y el cumplimiento de los requisitos especificados, por medio de la revisión del documento de especificación de requisitos. Este documento será revisado por un equipo de personas formado por usuarios, clientes y desarrolladores.

Para la correcta validación de requisitos es esencial que estos estén definidos correctamente y que permitan contestar a alguna pregunta con total claridad, el tipo de preguntas que suelen contestar serían:

- ¿El requisito está definido de manera clara y concisa?
- ¿El requisito tienen asignado algún nivel de prioridad?
- ¿El requisito está definido cuantitativamente y es medible?
- ¿Existe alguna relación con los demás requisitos?
- ¿El requisito forma parte de algún conjunto de requisitos?
- ¿El requisito incumple alguna otra restricción del proyecto?
- ¿Existe alguna posibilidad de interpretación errónea del requisito?

Cada una de estas preguntas debe poder ser contestada de manera inequívoca, obligando a los desarrolladores a volver a especificar el requisito en caso contrario.

1.2.5. Gestión de requisitos

La gestión de requisitos es la fase que realiza el control de cada una de las modificaciones que sean realizadas en el proyecto, revisando el documento de especificación de requisitos del sistema y garantizando sus actualizaciones y correcciones, entendiendo que estos requisitos son dinámicos y pueden ser

alterados a lo largo de cualquier ciclo de la vida de desarrollo del proyecto.

El equipo de desarrollo deberá especificar previamente las actividades que englobarán el proceso de gestión de requisitos y la manera de realizar dicha gestión. Cada proyecto llevará asociado una gestión diferente dependiendo de su complejidad y la naturaleza del equipo de desarrollo del proyecto.

Este procedimiento de gestión suele tener la siguiente información:

- Reglas sobre cómo establecer o modificar los requisitos en el proceso.
- Reglas y técnicas para controlar las versiones de cada requisito de la documentación.
- Reglas de seguimiento sobre los cambios del proyecto durante el proceso de desarrollo.
- Estado de cada uno de los requisitos y sus modificaciones.
- Definición de las personas encargadas de modificar los requisitos en cada caso.
- Reglas de seguimiento e información del estado de los requisitos.

1.3. Diseño

En cualquier desarrollo de *software* es muy importante seguir alguna especificación que indique a los desarrolladores qué etapas del proceso deben realizar y de qué manera, para que las acciones realizadas sean más coherentes y formales.

El diseño del *software* define las técnicas y principios que se llevarán a cabo para desarrollar un producto de *software*, suficientemente detallado como para permitir su desarrollo e implementación. El diseño del *software* pasa por las siguientes etapas:

1. **Diseño de la arquitectura:** define la relación entre cada uno de los

elementos estructurales del *software*.

2. **Diseño de procedimientos:** describe las funciones necesarias para realizar todos los procesos de información del *software*.
3. **Diseño de la interfaz:** describe cómo se puede comunicar el *software* con el usuario.

1.3.1. Modelos para el diseño de sistemas

Los modelos de diseño son un conjunto de pasos que permiten al desarrollador describir todos los aspectos del producto a desarrollar.

El diseño de sistemas debe implementar todos los requisitos definidos anteriormente en la fase de análisis, además de servir de guía para desarrollar todos los procedimientos que resuelvan estos requisitos.

Los modelos de diseño deben establecer criterios que permitan garantizar una calidad del producto, algunos de ellos son:

- **Diseño modular:** diferenciando los elementos independientes que realicen funciones específicas, y la relación entre ellos.
- **Diseño de datos:** se deben describir las relaciones entre las entidades de datos y los procedimientos asociados a estos. Utilizar modelos de diseño de datos como la Entidad-Relación.
- **Diseño jerárquico:** se debe establecer una jerarquía de prioridades entre los diferentes módulos ordenándolos por importancia.
- **Diseño de métodos:** debe describir de manera inequívoca los métodos necesarios para el desarrollo adecuado del producto.
- **Diseño de salida:** debe describir adecuadamente las interfaces con el usuario y las iteraciones de esta con el sistema, los mensajes y resultados de salida, determinando qué tipo de información se debe mostrar, de qué manera se realizará y el formato.
- **Diseño de documentos y ficheros:** se detallará el tipo de documentos que manejará la aplicación, su formato y el contenido de la información que contendrán, el tamaño de estos y su estructura de almacenamiento.

1.3.2. Diagramas de diseño. El estándar UML.

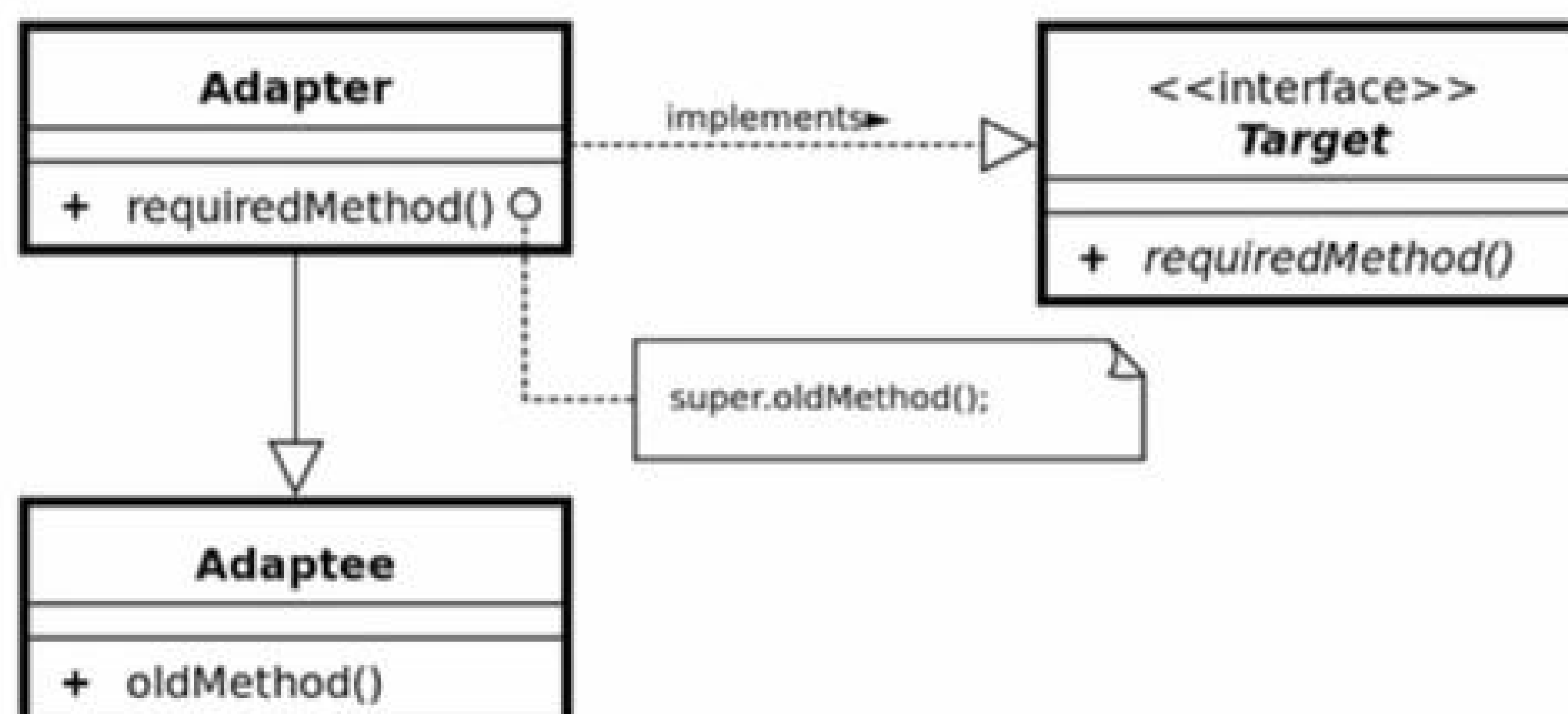
Los diagramas de diseño son instrumentos que nos permiten realizar un mapa conceptual del diseño del *software* por medio de esquemas gráficos. Es el equivalente al plano de una construcción o artefacto. Estos diagramas nos permiten planificar y desarrollar todas las características del proyecto.

El Lenguaje Unificado de Modelado (UML) es un lenguaje visual de modelado con sentido semántico para favorecer la arquitectura del diseño de un *software*. Normalmente se utiliza en grandes proyectos complejos de *software*. El objetivo del lenguaje es describir la estructura y comportamiento del sistema, así como los objetos que lo contiene.



El modelo utiliza elementos gráficos para representar el sistema y su comportamiento mediante una sintaxis y un conjunto de reglas. Debemos entender que no se trata de un lenguaje de programación sino de diseño que creará las pautas para el posterior desarrollo por medio de otros lenguajes.

El lenguaje UML está muy vinculado a la programación orientada objetos y sus elementos describen los elementos de este paradigma de programación.



- **Diagrama de clases:** describen los objetos que son los elementos básicos del lenguaje de programación, con una serie de propiedades y métodos que los definen.
- **Diagramas de objetos:** un objeto es una instancia de una clase, es decir la concreción de una clase particular con una serie de propiedades definidas.
- **Diagrama de casos de uso:** consiste en una descripción de las acciones de un sistema desde el punto de vista de un usuario.
- **Diagrama de estados:** describe los estados de cada objeto en un momento particular del proceso del *software*.
- **Diagrama de secuencias:** muestra la interacción entre las clases en base a tiempos.
- **Diagrama de actividades:** describe las diferentes actividades de las clases del sistema que realizan algún cambio en el mismo.
- **Diagrama de componentes:** describe la organización de los componentes físicos de un sistema.
- **Diagrama de distribución:** describe la arquitectura física de un sistema informático.

1.3.3. Documentación

El Consejo de Colegios de Ingeniería Informática ha elaborado una norma técnica para normalizar la elaboración de documentación en proyectos de software: la Norma Técnica CCII-N2016-02 para la realización de la Documentación de Proyectos en Ingeniería Informática.



Podemos descargar la norma completa en la siguiente dirección:

<https://www.cci.es/norma?view=form>



A continuación, veremos los diferentes apartados que regula la norma para elaborar documentación en un proyecto de *software*:

1. Introducción.

2. **Objeto del proyecto:** apartado para describir el objetivo del proyecto de manera de forma clara y concisa.

3. **Antecedentes:** relatamos la historia del proyecto, los pasos previos que nos han llevado hasta la propuesta de desarrollo del presente proyecto, enumeración de los hechos relevantes para conocer los antecedentes del proyecto (como las versiones anteriores en el caso de que las haya), las alternativas analizadas al presente proyecto y los motivos de la solución final propuesta.

4. **Descripción de la situación actual:** descripción de todos los elementos actuales que condicionen o se vean afectados por el proyecto en el ámbito de la organización o cliente, como los recursos humanos formación, equipamiento tecnológico, etc.

4.1. **Descripción del entorno actual:** descripción de la situación actual de los sistemas que se verán afectados por el proyecto, como la organización del trabajo, infraestructura de la organización, etc.

4.2. **Resumen de las principales deficiencias identificadas:** se trata de

describir las deficiencias analizadas en el sistema previamente al proyecto y que este subsanará.

5. **Normas y referencias:** enumeración de la normativa, reglamentos, directrices y referencias que se utilizaron en la elaboración del proyecto.

5.1. **Disposiciones legales y normas aplicadas:** descripción de las disposiciones legales, reglamentos, ordenanzas, etc., aplicables al proyecto, entre las que normalmente se pueden señalar la ley de protección de datos de carácter personal (LOPD), las leyes de seguridad o accesibilidad (ENI, ENS), las normas de calidad (ISO), etc.

5.2. **Bibliografía:** se describirá todos los textos utilizados para el contenido del proyecto, como libros, artículos, revistas, webs, y otros textos, describiendo los datos de su bibliografía como el título, autor, enlace, IBSM, etc.

5.3. **Métodos y herramientas:** descripción de los métodos, herramientas, modelos, métricas y prototipos utilizadas para desarrollar algún elemento del proyecto.

5.3.1. **Métodos y Herramientas:** se describirán los métodos y herramientas utilizados para realizar los diferentes cálculos, estimaciones o estadísticas del proyecto.

5.3.2. **Modelos, Métricas y Prototipos:** se describirán los modelos, métricas o prototipos utilizados para elaborar los diferentes cálculos, funciones o componentes del sistema.

5.4. **Mecanismos de control de calidad aplicados durante la redacción del proyecto:** debemos describir los procesos específicos para garantizar la calidad en la elaboración de la documentación del proyecto utilizando la norma técnica CCII-N2016-02, generando un documento de manera clara, concisa y entendible para todos los usuarios.

5.5. **Otras referencias:** añadiremos otras referencias a mayores de la bibliografía y que estén relacionadas de alguna forma con el proyecto, como la enumeración de proyectos similares, o referencias que inciden indirectamente en el proyecto de alguna manera.

6. **Definiciones y abreviaturas:** enumeración de la terminología empleada en el proyecto, como las definiciones, tecnicismos, abreviaturas, etc.

7. **Requisitos iniciales:** establecimiento de los requisitos necesarios para desarrollar el proyecto, las especificaciones del sistema y los métodos de validación para comprobar que se han cumplido todas las características del producto.

8. **Alcance:** en este apartado proporcionaremos un marco acerca de todas las entregas que se realizarán al cliente con el proyecto final, precisando su contenido y características.

9. **Hipótesis y restricciones:** relataremos todas condiciones de salida y las limitaciones que se hayan identificado para la ejecución del proyecto y que tengan incidencia en cualquiera de los aspectos del proyecto, como el coste, plazos de entrega calidad del producto, etc.

10. **Estudio de alternativas y viabilidad:** enumeración de todas las alternativas que se han estudiado a la hora de plantearse el proyecto, las características de cada una de ellas, y la justificación de la elección de la alternativa adoptada.

11. **Descripción de la solución propuesta:** descripción concisa de la propuesta planteada y las razones para haberla elegido, descripción de las características principales de la solución propuesta.

12. **Análisis de Riesgos:** enumeración de los riesgos del proyecto, lista de riesgos clasificada por importancia, redacción de un plan de riesgos.

13. **Organización y gestión del proyecto:** este apartado describirá todo lo referente a la organización del trabajo del equipo de desarrolladores en el proyecto, enumerando las directrices y métodos de trabajo, normas de seguimiento, métodos de comunicación entre los miembros, protocolos para la gestión de cambios, etc. Los detalles de la organización estarán organizados de la siguiente manera:

13.1. Organización

13.1.1. **Actores del proyecto y relaciones entre los mismos:** descripción del personal que llevará a cabo el proyecto, el rol de

cada uno y las relaciones y jerarquía entre ellos.

13.1.2. **Estructura interna:** denominación de los responsables, coordinadores y otro tipo de actores en el equipo de trabajo de cada grupo.

13.1.3. **Interfaces externas:** en este apartado se analiza la posibilidad de integrar el producto en sistemas ya implementados, y las características de estos sistemas para facilitar la integración.

13.1.4. **Roles y responsabilidades:** se realizará una relación de roles y responsabilidades de cada uno entre todos los actores del proyecto.

13.2. **Gestión del proyecto:** se analizarán diferentes aspectos de la gestión del proyecto como:

- Gestión de requisitos.
- Gestión y validación de las funcionalidades requeridas.
- Gestión y validación del diseño técnico y arquitectura del sistema.
- Gestión de la configuración.
- Gestión de incidencias.
- Gestión de riesgos.
- Gestión y validación de las pruebas.
- Gestión de la coordinación del proyecto.
- Gestión de plazos y presupuesto.
- Gestión y aseguramiento de la calidad.
- Gestión y validación de los entregables.

14. **Planificación temporal:** en este apartado detallaremos el plazo de ejecución y entrega del proyecto. Mediante un cronograma o calendario de actividades explicaremos las entregas de partes o todo el producto. La duración

de cada etapa de desarrollo y cualquier otro aspecto relacionado con la temporalidad.

14.1. Evolución del plan de proyecto: descripción del seguimiento del ciclo de vida del producto, desde el inicio del proyecto hasta su entrega final, enumeración de todos los cambios e incidencias en el desarrollo del producto que afecten a su temporalidad.

15. Resumen del Presupuesto: en este apartado describiremos el coste total del proyecto, desglosando los diferentes conceptos por partidas.

16. Orden de prioridad de los documentos básicos del proyecto: realizaremos una descripción del orden de los diferentes documentos que se aporten para el proyecto, organizándolos por prioridades según las especificaciones del proyecto, el presupuesto y el diseño del mismo

17. Anexos: se trata de documentos básicos sobre el proyecto, que describen detalladamente partes del proceso mediante análisis, métricas, estimaciones, convenientemente redactados y organizados en un índice.

17.1. Documentación de entrada: incluirá toda la documentación de entrada relevante para el proyecto como la Petición de Propuesta, Pliego de Condiciones, información de fases anteriores, etc.

17.2. Análisis y diseño del Sistema: esta información dependerá del modelo de desarrollo de *software* elegido, describiremos el diseño del modelo a desarrollar, la arquitectura el sistema propuesto, el diseño de las funcionalidades, interfaces y flujo de datos, el análisis de los casos de uso, y todos los diagramas que se consideren importantes.

17.3. Estimación de tamaño y esfuerzos: detallaremos las métricas que utilizaremos en el proyecto utilizando los estándares convenientes para la normalización de las métricas, deberán estar documentadas y referenciadas.

17.4. Planes de gestión del proyecto: según la norma técnica del proyecto, utilizaremos los estándares de normas internacionales para llevar a cabo la documentación de este anexo:

- «Guía de los Fundamentos para la Dirección de Proyectos» o UNE.

- ISO 21500:2013 «Directrices para la dirección y gestión de proyectos».
- Guía del PMBOK.

17.4.1. **Gestión de la integración:** se describen procesos como el desarrollo del acta de constitución del proyecto, el plan de dirección del mismo, el de ejecución, seguimiento y control de cambios y el de cierre del proyecto.

17.4.2. **Gestión del alcance:** detallaremos todos los procesos que influyan indirectamente en el proyecto para que se desarrolle adecuadamente. Estará compuesto de:

- Planificar la gestión del Alcance.
- Recopilar requisitos.
- Definir el alcance.
- Crear EDT,
- Validar el alcance,
- Controlar el alcance,

17.4.3. **Gestión de plazos:** detallar los procesos relacionados con el cumplimiento de los plazos establecidos para el desarrollo del proyecto. Estará compuesto por los siguientes procesos:

- Planificar la gestión del cronograma.
- Definir las actividades.
- Secuenciar las actividades.
- Estimar los recursos de las actividades.
- Estimar la duración de las actividades.
- Desarrollar el cronograma.

- Controlar el cronograma.

17.4.4. **Gestión de costes:** detallar todos los procesos en la estimación de costes y presupuestos necesarios para calcular el precio final del proyecto. Está compuesto por los siguientes procesos:

- Planificar la gestión de los costes.
- Estimar los costes.
- Preparar el presupuesto de costes.
- Controlar los costes..

17.4.5. **Gestión de la calidad:** incluye todos los procesos involucrados en la calidad del producto final. Está compuesto por los siguientes procesos:

- Planificar la gestión de calidad.
- Realizar el aseguramiento de calidad.
- Controlar la calidad.

17.4.6. **Gestión de recursos humanos:** incluye todos los procesos para organizar y dirigir los recursos humanos implicados en el proyecto. Está formado por los siguientes procesos:

- Planificar la Gestión de los RRHH.
- Adquirir el equipo del proyecto.
- Desarrollar el equipo del proyecto.
- Dirigir el equipo del proyecto.

17.4.7. **Gestión de comunicaciones:** describirá todos los procesos relacionados con el flujo de la información, las comunicaciones y la transferencia de datos. Está compuesto por los siguientes procesos:

- Planificar la gestión de las comunicaciones.

- Gestionar las comunicaciones del proyecto.
- Controlar las comunicaciones.

17.4.8. **Gestión de riesgos:** incluiremos todos los procesos relacionados con la gestión de riesgos. Está compuesto por los siguientes procesos:

- Planificar la gestión de riesgos.
- Identificar los riesgos.
- Analizar cualitativamente los riesgos.
- Analizar cuantitativamente los riesgos..
- Planificar la respuesta a los riesgos
- Controlar los riesgos.

17.4.9. **Gestión de adquisiciones:** se relacionarán todos los procesos necesarios para adquirir todos los productos y servicios necesarios para el desarrollo del proyecto. Se compone de los siguientes procesos:

- Planificar la gestión de las adquisiciones.
- Efectuar las adquisiciones.
- Controlar las adquisiciones.
- Cerrar las adquisiciones.

17.4.10. **Gestión de interesados (*Stakeholders*):** añadir los procesos para identificar y gestionar las necesidades y expectativas de los usuarios, clientes y patrocinadores del proyecto incluidos terceros como los proveedores, público objetivo, etc. Se compone de los siguientes procesos:

- Identificar a los interesados.
- Planificar la gestión de los interesados.

- Gestionar la relación con los interesados.
- Controlar la relación con los interesados.

17.5. Plan de seguridad: esta sección reflejará todas las medidas, metodologías y herramientas que se deben utilizar para aplicar las normas de seguridad necesarios para garantizar el proyecto, así como los puntos críticos y posibles debilidades donde la seguridad es determinante.

17.6. Otros anexos: esta sección es opcional dependiendo si necesitamos incluir alguna información adicional no contemplada en los anteriores apartados para la realización del proyecto.

18. Especificaciones del sistema: este apartado incluirá toda la información detallada de los requisitos del proyecto, los diagramas utilizados (casos de usos, UML, modelo relacional de datos, etc.). Es una parte de la documentación muy importante ya que debe servir de base al análisis y diseño del sistema.

19. Presupuesto: otra parte crucial de la documentación que determina y justifica el coste del proyecto y el precio final del producto. Está compuesto por los siguientes procesos:

- Cuadro de precios de las unidades de medida correspondientes: elementos de *hardware*, elementos de *software*, horas por trabajador de diferentes categorías, elementos auxiliares y otros.
- Valoración económica global, desglosando cada uno de los componentes.

La norma técnica CCII-N2016-02 incluye una tabla de control de seguimiento de la documentación del proyecto que incluimos a continuación:

Control de la documentación del proyecto

5. Control de la documentación del proyecto

Hoja de Control de la Documentación del Proyecto	Presentado Si / No	Justificación de la exclusión
ÍNDICE		Obligatorio
MEMORIA		Obligatorio
1. Introducción		Obligatorio
2. Objeto del proyecto		Obligatorio
3. Antecedentes		Recomendable si existen
4. Descripción de la situación actual		Obligatorio
4.1. Descripción del entorno actual		Obligatorio
4.2. Resumen de las deficiencias identificados		Obligatorio
5. Normas y referencias		Obligatorio
5.1. Disposiciones legales y normas aplicadas		Obligatorio
5.2. Bibliografía		Opcional
5.3. Métodos, Herramientas, Modelos, Métricas y Prototipos		Obligatorio
5.3.1. Métodos y Herramientas		Obligatorio
5.3.2. Modelos, Métricas y Prototipos		Obligatorio
5.4. Mecanismos de control de calidad aplicados durante la redacción del proyecto		Obligatorio
5.5. Otras referencias		Opcional
6. Definiciones y abreviaturas		Recomendable
7. Requisitos iniciales		Obligatorio
8. Alcance		Obligatorio
9. Hipótesis y restricciones		Recomendable
10. Estudio de alternativas y viabilidad		Obligatorio
11. Descripción de la solución propuesta		Obligatorio
12. Análisis de Riesgos		Obligatorio
13. Organización y gestión del proyecto		Obligatorio
13.1. Organización		Obligatorio
13.2. Gestión del proyecto		Obligatorio
14. Planificación temporal		Obligatorio
15. Resumen del Presupuesto		Obligatorio
16. Orden de prioridad de los documentos básicos		Obligatorio
17. ANEXOS		Obligatorio
17.1. Anexo - Documentación de entrada		Obligatorio
17.2. Anexo - Análisis y Diseño del Sistema		Obligatorio
17.3. Anexo - Estimación de Tamaño y Esfuerzos		Obligatorio
17.4. Anexo - Planes de Gestión del proyecto		Obligatorio
17.4.1. Gestión de la integración		Justificar su exclusión
17.4.2. Gestión del alcance		Justificar su exclusión
17.4.3. Gestión de plazos		Obligatorio
17.4.4. Gestión de costes		Obligatorio